

Group Project Specification

University Course Registration, Timetable & Attendance Management System

1. Project Overview

Your group will design and implement a console-based University Course Registration, Timetable & Attendance Management System in C++. The system allows students to register for courses subject to prerequisite and capacity rules, automatically detects timetable clashes, lets lecturers manage course offerings and record class attendance, and lets administrators oversee the whole system. All data must persist between program runs through file storage.

Attendance capture is deliberately open-ended: the specification fixes the attendance core (sessions, records, validation, and reporting) but requires you to implement the capture mechanism behind an abstract AttendanceCapture interface and choose your own concrete mechanism from an approved list (Section 3.7). Your choice, and its justification, form part of the design checkpoint.

The purpose of this project is not merely to produce a working program, but to demonstrate disciplined object-oriented design. You will first design your system as a UML class diagram, defend that design at a checkpoint, and then implement it. Your final code will be checked for conformance against your approved design, and any deviations must be justified in a design change log.

3. System Description and Functional Requirements

The system must support three user roles: Student, Lecturer, and Administrator. Each role logs in and sees a role-specific menu. The minimum functional requirements are listed below; groups are encouraged to add well-designed extensions.

3.1 User Management

- FR1.1 – The system shall support login for three user roles: Student, Lecturer, and Administrator.
- FR1.2 – The Administrator shall be able to create, update, and remove user accounts.
- FR1.3 – Each role shall see a different dashboard/menu determined polymorphically (not by chains of if/else on a role string).

3.2 Course Management

- FR2.1 – The Administrator shall be able to create, edit, and remove course offerings, each with a code, title, credit value, capacity, assigned lecturer, and prerequisites.
- FR2.2 – The system shall support at least three course types (e.g., lecture-based, lab-based, and project-based courses) with differing credit or grading behaviour implemented through a class hierarchy.
- FR2.3 – A Lecturer shall be able to view the enrolment list of their own courses only.

3.3 Enrolment Engine

- FR3.1 – A Student shall be able to enrol in and drop courses.
- FR3.2 – Enrolment shall be rejected with a meaningful message when prerequisites are not met or the course is full.

- FR3.3 – These rule violations shall be signalled using custom exception classes.
- FR3.4 – Prerequisite chains (a course whose prerequisite has its own prerequisite) shall be checked correctly.

3.4 Timetable and Clash Detection

- FR4.1 – Each course shall have one or more weekly time slots (day, start time, end time, location).
- FR4.2 – The system shall reject an enrolment that creates a timetable clash with the student's existing courses.
- FR4.3 – A Student shall be able to view their personal weekly timetable.
- FR4.4 – Clash detection between two time slots shall be implemented via an overloaded operator (e.g., operator== or a dedicated overlaps operator).

3.5 Persistence

- FR5.1 – All users, courses, and enrolments shall be saved to files and reloaded on startup, preserving all relationships between objects.
- FR5.2 – The file format is your design decision, but loading and saving must be implemented through a clean, encapsulated interface (e.g., a Repository or Storage class), not scattered file code.
- FR5.3 – Corrupt or missing data files shall be handled gracefully using exceptions.

3.6 Reporting

- FR6.1 – The Administrator shall be able to generate at least one report (e.g., enrolment summary per course).
- FR6.2 – Printing of domain objects (students, courses, timetables) shall use an overloaded stream insertion operator (operator<<).

3.7 Attendance (abstract capture with group choice)

Attendance core (mandatory, identical for all groups). Every group implements the same attendance domain model:

- FR7.1 – A Lecturer shall be able to open an attendance session for one of their own course time slots and close it manually or by expiry (configurable duration, e.g., 10 minutes).
- FR7.2 – A student may be marked present only if the session is open, the student is enrolled in that course, and the student has not already been marked in that session. Each violation shall raise a distinct custom exception (e.g., SessionClosedException, NotEnrolledException, DuplicateAttendanceException).
- FR7.3 – Attendance records shall be immutable: a record stores the student, session, timestamp, status (present/late), and how it was captured. Corrections are made by appending a correction record (with the acting lecturer and reason), never by editing or deleting.
- FR7.4 – The system shall report per-student and per-course attendance percentages. (Optional extension: an eligibility report listing students below a configurable threshold, default 80%.)
- FR7.5 – Attendance data shall persist and reload with all other system data.

Capture mechanism (group choice via an abstract interface). How a 'present' event enters the system must be hidden behind an abstract class:

- FR7.6 – The group shall define an abstract AttendanceCapture class with pure virtual functions (e.g., beginSession(), captureNext(), endSession()). The attendance core shall depend only on this interface — it must not know which concrete mechanism is in use.
- FR7.7 – The group shall implement at least TWO concrete capture classes: (a) FileReplayCapture — mandatory for all groups — which replays capture events from a text file (used for testing and bulk

demonstration, including malformed lines that trigger exceptions); and (b) ONE interactive mechanism chosen by the group from the approved list below.

- FR7.8 – The active capture mechanism shall be selectable at runtime (e.g., a lecturer setting), demonstrating that concrete captures are interchangeable through base-class pointers/references.

Approved interactive capture mechanisms (choose one):

- Option A – Rotating session code: the system displays a short random code with an expiry; students enter the code from their own menu to check in.

Your chosen option must be named and justified in the D1 design rationale, and your UML must show the AttendanceCapture hierarchy. Optional bonus (up to +5 marks, capped at 100): a second interactive capture mechanism, or real hardware/webcam capture (e.g., OpenCV/ZBar QR scanning), provided the interface design is unchanged.

4. Mandatory OOP and Technical Requirements

Regardless of extensions, your implementation must demonstrably include all of the following. Each item will be inspected in the code review and probed at the viva.

Concept	Minimum requirement
Encapsulation	No public data members in domain classes; all state accessed through a deliberate public interface. Business rules enforced inside classes, not in the menu code.
Inheritance & polymorphism	At least two inheritance hierarchies (e.g., Person and Course) with an abstract base class, virtual functions overridden in derived classes, and at least one place
	where objects are used polymorphically through base-class pointers or references. Base classes must declare virtual destructors.
Abstract interfaces & dependency inversion	The AttendanceCapture abstract class (pure virtual functions, virtual destructor) with at least two concrete implementations, selected and used at runtime through base-class pointers/references. The attendance core must compile without knowledge of any concrete capture class.
Composition / aggregation	At least one composition relationship (e.g., Course owns its AttendanceRegister and Timetable owns TimeSlot objects) and one aggregation relationship (e.g., Course refers to enrolled Students), and you must be able to explain the difference in your design.
Constructors / destructors / Rule of Three or Five	At least one class manages a dynamic resource and correctly implements the destructor, copy constructor, and copy assignment operator (or explicitly deletes copying with justification).
Operator overloading	At least three meaningful overloaded operators, including operator<< for output and one comparison/logic operator used in clash detection.

Exception handling	A small hierarchy of custom exception classes derived from <code>std::exception</code> , thrown by the domain layer and caught at the user-interface layer.
Templates & STL	Appropriate use of STL containers (vector, map, etc.) and at least one groupwritten class or function template (e.g., a generic <code>Repository<T></code>).
File handling	Object state saved and restored through <code>fstream</code> , encapsulated behind a storage interface.
Static members / constcorrectness	At least one justified use of a static member; getter functions and read-only parameters declared <code>const</code> .
Code organisation	Each class split into a header (.h) and implementation (.cpp) file with include guards; the project builds with a provided Makefile or CMakeLists.txt using <code>g++ std=c++17 -Wall</code> .

5. Group Roles and Responsibilities

Each group has exactly three members. The suggested division below gives every member both design and implementation responsibility. You may adjust boundaries, but the final report must contain a contribution table stating who designed, implemented, and tested each class, and every member must be able to explain the whole design at the viva.

Member	Responsibility
Member 1 – Domain model	Person hierarchy (Student, Lecturer, Administrator), Course hierarchy, enrolment rules, custom exception hierarchy.
Member 2 – Scheduling & attendance engine	TimeSlot and Timetable classes, operator-based clash detection, prerequisite checking, the attendance core (AttendanceSession, AttendanceRecord, AttendanceRegister, validation exceptions), the abstract AttendanceCapture interface with the group's chosen concrete capture, and attendance reporting.
Member 3 – Persistence & UI	Storage/Repository classes and file formats (including attendance persistence and the FileReplayCapture), template <code>Repository<T></code> if used, menu system, integration, build scripts, and system testing.